

Uber & Lyft Cab Prices Prediction

Authors: Aaron Perkey and Dillon Kim

1. Problem Statement

The rise of ride-sharing services like Uber and Lyft has led to fare prices that fluctuate significantly depending on the time, location, and weather. Our project set out to predict cab ride prices using machine learning models by analyzing both ride data (like distance, source, and destination) and weather data (such as temperature and visibility). We were curious to see whether we could accurately forecast prices and offer practical insights to help users or businesses choose the most cost-effective times to ride.

2. Models Used

We tried out two machine learning models that approach the problem differently:

- **Linear Regression:** This was our starting point because it's straightforward to interpret. It assumes a linear connection between the input features and the fare price.
- **K-Nearest Neighbors (KNN) Regression:** This model doesn't make assumptions about data distribution. Instead, it looks at the average fare of the 'k' most similar past rides to predict a new fare.

Both models were coded in Python using scikit-learn, and we cleaned and scaled the features before training. Our dataset combined cab ride records (`cab_rides.csv`) and weather observations (`weather.csv`).

3. Performance Comparison and Justification

We used two metrics to evaluate how well the models performed:

- **Root Mean Squared Error (MSE):** How far off the predictions were from actual prices, on average (Square Root). In this metric, we want smaller values when deciding the model.
- **R-squared Score (R2):** How well the model explained the variation in fare prices. In this metric we want larger values when deciding the model.

Results:

- The Linear Regression model gave us a fair starting point, but its R2 score wasn't very strong. It struggled with capturing more complex fare behaviors.
- The KNN Regression model did better across the board. Once we fine-tuned the number of neighbors using cross-validation, the performance noticeably improved. This confirmed our assumption that ride prices don't always follow a linear pattern and that local comparisons (as done by KNN) are more effective here.

Based on these results, KNN turned out to be the better fit for this problem.

KNN R2= 0.965

Linear Regression R2 = 0.928

KNN RMSE = 1.75

Linear Regression RMSE = 2.50

4. Pros and Cons of Our Approach

Pros:

- Combining ride and weather data made our predictions more informed and realistic.
- Testing both a simple and a more adaptive model gave us insight into the strengths and weaknesses of each.
- Preprocessing steps like feature scaling helped KNN perform accurately by ensuring fair distance comparisons.

Cons:

- KNN can be slow on large datasets and is sensitive to the value of 'k'.
- We didn't apply regularization in the linear regression model, which could have helped reduce overfitting or improve accuracy.
- We stuck with basic models, but exploring more advanced techniques like Random Forests or Gradient Boosting might have improved results even more.

In the end, our project offered valuable hands-on experience in modeling real-world data. It also showed that models like KNN, which adapt to local patterns, can be beneficial when dealing with complex, real-world pricing data.